Situation engineering for evidence-applicable artificial intelligence

Geunsik Lim[1*]

[1] Data Service Laboratory, Samsung Electronics, Seoul, South Korea .

Corresponding author(s). E-mail(s): geunsik.lim@samsung.com;

Abstract

Artificial-intelligence systems increasingly combine prompts, retrieved context, memory, tools and agent harnesses, yet these components do not by themselves determine whether an available fact applies to the question being answered. A statement may be well supported and still be wrong because it is stale, merely proposed, outside the relevant scope, unknown to the queried observer or true only in another possible world. We call this failure *situation blindness* and define *situation engineering* as the explicit construction, updating, validation and exposure of the query-relative world state that governs evidence applicability. We introduce Situation-Catching Question Answering (SCQA), a reference control architecture that separates situation sensing, state update and response policy, and SituationCatch-Bench, a 4,200-instance diagnostic suite covering temporal change, hidden premises, source conflict, modality, scope, observer knowledge and counterfactual worlds. Under oracle structured states, SCQA satisfies all generated invariants, whereas non-situational rules fail in predictable categories. Replacing gold states with states inferred from claim text reduces accuracy from 100.0% to 77.9%, and deliberately corrupted states reduce it to 35.1%, exposing situation sensing as a distinct bottleneck. We interpret these controlled results as software-level diagnostics, not as evidence of natural-text or end-to-end language-model performance. The main contribution is therefore a falsifiable problem definition, a modular evaluation decomposition and a reproducible research scaffold for measuring where situation sensing, state maintenance, policy selection and generation fail. By locating situation engineering alongside prompt, context, retrieval, memory, tool, harness, evaluation and serving engineering, the framework provides researchers with interfaces, reporting standards and open experiments for developing systems that distinguish relevant evidence from evidence that is actually applicable.

Keywords: large language models, situation engineering, evidence applicability, situation awareness, hallucination, trustworthy artificial intelligence, temporal reasoning, question answering, agent systems

Large language models (LLMs) increasingly answer questions by combining parametric knowledge with retrieved documents, tools, memory and long conversational histories. These mechanisms improve evidence coverage, but they do not guarantee that a system identifies the *situation* in which a fact is valid. A model may read that a person was appointed and later resigned, yet still name that person as the current office holder. It may turn a proposal into a decision, extend a local policy to an entire organization, answer from information unavailable to the queried observer, or collapse a hypothetical scenario into the real world. In each case the decisive evidence can already be present. The failure is not simply missing knowledge; it is incorrect evidence applicability.

We define *situation blindness* as producing an answer or action that is supported by at least one available claim but inconsistent with the time, scope, modality, observer knowledge or possible world required by the query. This category matters because conventional factuality checks can mark the supporting claim as true while missing that it is true in the wrong situation. Natural question-answering resources already show that a material subset of ordinary questions depends on extra-linguistic time or place, and that updated evidence does not eliminate the resulting performance loss [5, 11–14].

Retrieval-augmented generation supplies external evidence [1]; self-reflective and tool-using systems add critique and action [2–4]; context engineering organizes the inference-time information payload [7]; and agent harnesses mediate tools, memory, permissions, verification and feedback [8, 9]. These layers are necessary, but none requires an explicit answer to a separate question: *which interpretation of the available evidence governs this interaction now?*

We call the corresponding discipline *situation engineering*: the systematic design of representations, sensing interfaces, update rules, validation procedures and response policies that establish the active query-relative world state before an AI system commits to an answer or action. For a query q and evidence E, a minimal state is

$$S(q, E) = \{\mathcal{A}, \mathcal{V}, \mathcal{T}, \mathcal{P}, \mathcal{M}, \mathcal{K}, \mathcal{W}, \mathcal{U}\}, \tag{1}$$

where the variables denote actors, events, valid time, scope, modality and source status, observer-specific knowledge, possible world and unresolved answer-critical information. A policy then selects

$$\pi(S) \in \{\textsc{Answer}, \textsc{Clarify}, \textsc{Retrieve}, \textsc{Abstain}, \textsc{Act}\}. \tag{2}$$

This decomposition separates evidence availability from evidence applicability.

The present paper makes four bounded contributions. First, it defines situation blindness as a testable failure class and relates it to established work on situations, temporal reasoning, belief state tracking and knowledge revision. Second, it specifies situation engineering as a systems contract connecting prompt, context, retrieval, memory, tool, harness, evaluation and serving engineering without subsuming them. Third, it introduces SCQA and SituationCatch-Bench as a transparent oracle-state diagnostic for causal unit tests and component ablations. Fourth, it provides a staged evaluation and reporting protocol for future end-to-end studies on natural language, multiple models, human clarification and deployed agents. We deliberately do not claim that the current rule-based implementation solves situation sensing in unrestricted text. Instead, the paper separates the solved control invariants from the open perception, learning and generalization problems so that future work can measure progress without conflating them.

Related conceptual foundations

Situation engineering draws on several mature traditions but is not identical to any one of them. Situation semantics treats meaning relative to partial situations [10]. Situation and event calculi formalize how actions change fluents over time [24–26]. Temporal reasoning provides interval relations and validity semantics [6]. Belief revision and truth-maintenance research asks how an agent should update inconsistent commitments [18, 19]. POMDPs and dialogue-state tracking maintain uncertainty over latent states for sequential decisions [20, 21]. Temporal knowledge graphs and bitemporal databases record changing facts and distinguish when a fact is valid from when it is stored [22].

The proposed contribution is a systems-level applicability contract for generative and agentic AI. It requires these representations to connect to natural-language evidence, provenance, unresolved alternatives and a response policy that can answer, retrieve, clarify, abstain or act. Thus the novelty is not a new universal logic. It is the explicit decomposition of modern AI reliability into (i) sensing a situation from heterogeneous evidence, (ii) maintaining a provenance-linked state, (iii) validating applicability and (iv) propagating unresolved state into behaviour. This framing allows existing logical, probabilistic and neural methods to be compared through a common set of interfaces and failure measurements.

Dynamic epistemic logic and common-ground modelling formalize belief-changing information and mutually recognized conversational commitments [16, 17]. Theory-of-mind models, world models and active perception further address attributed beliefs, latent environment state and information acquisition. These traditions reinforce rather than eliminate the need for careful attribution: situation engineering does not claim their theoretical results as new. Its proposed novelty is the systems contract that exposes their state variables to provenance-aware answer/retrieve/clarify/abstain gates in contemporary generative systems.

Situation Engineering framework

Prompt engineering specifies a task; context and retrieval engineering select information; memory engineering retains observations; tools expose capabilities; and harness engineering controls execution. Situation engineering addresses a narrower semantic

Table 1 Typical coverage of neighbouring state-based research traditions. Entries summarize common formulations rather than claiming that every method in an area has the same capabilities.

Area	Explicit state	Time	Observer	Worlds	Uncertainty	Answer/action gate	Provenance
Situation/event calculus	✓	✓	limited	partial	usually no	action-centred	limited
Belief revision / truth maintenance	✓	partial	partial	partial	partial	no	limited
Dialogue-state tracking / POMDPs	✓	sequential	✓	partial	✓	✓	uncommon
Temporal KG / bitemporal data	✓	✓	limited	uncommon	limited	no	✓
RAG / tool agents	often implicit	limited	uncommon	uncommon	model-derived	partial	partial
Proposed situation interface	✓	✓	✓	✓	required	✓	✓

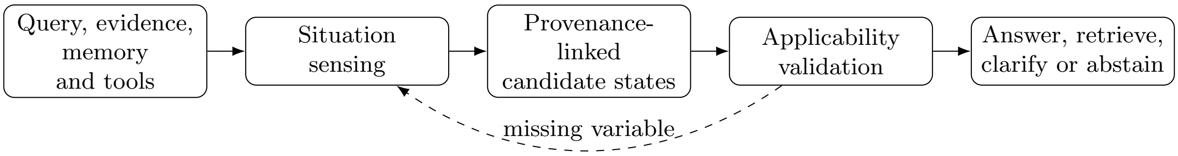

Fig. 1 Proposed situation-engineering interface. The taxonomy is a conceptual systems framework; the present experiments validate only the controlled state update and response-policy components.

question: which available claims apply to this query, observer and world now? It is therefore a proposed interface among existing components, not a replacement name for them and not an empirically validated universal layer.

For query q, evidence E, retained memory M_t, tool observations O_t and harness trace H_t, situation construction produces candidate states $\mathcal{S}_t$ with unresolved variables U_t and provenance P_t:

$$(q, E, M_t, O_t, H_t) \to (\mathcal{S}_t, U_t, P_t). \tag{3}$$

Each evidence claim is assigned a query-relative applicability value

$$\text{Apply}(e, q, S_t) \in \{\text{valid}, \text{invalid}, \text{unresolved}\}. \tag{4}$$

Preserving *unresolved* is essential. A system should retrieve, clarify or abstain when answer-critical state cannot be established rather than silently converting uncertainty into a plausible binary judgement.

Operations and component boundaries

Six operations define the interface. *Sensing* extracts actors, events, times, modality, source status, scope, observer knowledge and world identity. *Assembly* links observations into candidate states. *Updating* applies supersession, cancellation and belief transitions. *Validation* checks applicability, conflicts, provenance and missing variables. *Policy* selects an answer, retrieval, clarification, abstention or action. *Observability* records the active state and its evidence.

These responsibilities do not absorb neighbouring disciplines. A retriever still owns ranking and freshness; a memory system owns retention; a tool layer owns invocation semantics; a harness owns permissions and recovery; and serving systems own resource efficiency. The proposed situation layer consumes their outputs and returns applicability decisions and unresolved-state signals. Likewise, longer context is not sufficient: it may contain an appointment and a later resignation, a proposal and a final decision, or real and hypothetical claims simultaneously. Applicability requires resolving their relation rather than merely retaining both.

SCQA as a bounded control implementation

Situation-Catching Question Answering (SCQA) instantiates the interface for seven controlled variables. Its deterministic sensor or supplied gold state feeds auditable update rules and an answer/clarify/abstain policy. SCQA has no learned parameters. It is designed to test whether removing a required variable produces the matching failure signature, not to compete with contemporary language models.

This scope matters for interpretation. The generated benchmark and solver share an ontology, making the oracle-state experiment analogous to a protocol conformance suite. The predicted-state condition introduces sensing error but still uses generated English. Natural-language generalization, comparison with strong prompting and retrieval systems, multilingual transfer, human annotation and deployment efficiency remain untested hypotheses. The released multi-provider and annotation programs make these studies executable but do not turn an unexecuted protocol into evidence.

Testable predictions

The framework yields falsifiable predictions for future matched experiments. Holding model, evidence and computation budget fixed, an explicit situation state should improve the categories whose decisive variables are otherwise implicit. Gains should diminish when a baseline already represents the same state. Predicted-state errors should decompose into sensing, update, policy and generation failures. Corrupting one state slot should selectively damage its matching category. Finally, the benefit must be evaluated jointly with token use, latency, retrieval calls, monetary cost and unnecessary clarification. Failure of these predictions would argue against treating situation engineering as a distinct practical interface.

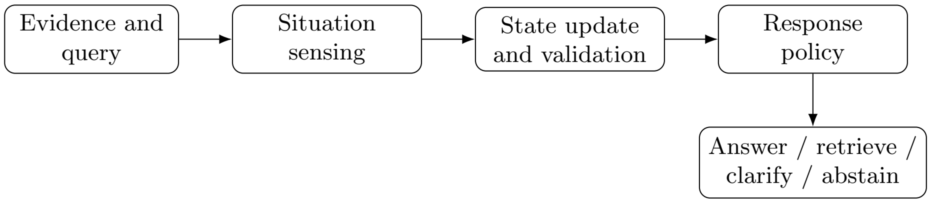

Fig. 2 Evaluation decomposition. The present experiment supplies oracle structured situation variables and evaluates state update and response policy. End-to-end research must additionally measure natural-language situation sensing.

Results

A controlled benchmark isolates seven applicability invariants

SituationCatch-Bench v0.1 contains 4,200 generated English instances, with 600 instances in each of seven diagnostic categories: temporal state change, omitted answer-critical premises, source conflict, proposed versus confirmed events, local versus universal scope, observer-dependent knowledge and real versus counterfactual worlds. Every item provides a question, shuffled claims, query time, required state variables, a gold action and, where answerable, a gold answer. The benchmark is intentionally transparent: each category is a software-level test of one applicability invariant rather than a claim to natural linguistic diversity.

Oracle-state control satisfies the generated invariants

SCQA was compared with lexical retrieval, recency-ranked retrieval and a latest-mention rule. All methods received the same generated claims; SCQA alone used the complete oracle situation state and transition semantics. SCQA obtained 100.0% action-and-answer accuracy. Latest mention obtained 85.7%, recency ranking 71.4% and lexical retrieval 70.0% (Fig. 3). These numbers are deterministic consequences of the benchmark and implementation and should be interpreted as regression-test coverage, not as estimates of model performance in a population of natural questions.

The 85.7% value is structurally induced: the categories are equally weighted and the latest-mention control systematically fails the hidden-premise category, so success on six categories yields $6/7 = 85.7\%$. The 14.3 percentage-point gap is therefore not presented as an empirical effect size against a competitive baseline.

The category-level pattern provides the useful result. Latest mention fails on hidden-premise items because it answers when clarification is required. Recency ranking fails when the newest global event is not known to the queried observer. Lexical overlap is vulnerable to weak but topically similar conflicting claims. Thus a single aggregate "hallucination" label hides distinct and testable mechanisms (Fig. 4).

Component ablations produce matched failure signatures

Removing temporal validity, source status, modality, scope, observer knowledge, world identity or missing-variable detection selectively damages the matching category (Fig. 5). This causal alignment is the main empirical contribution of the controlled

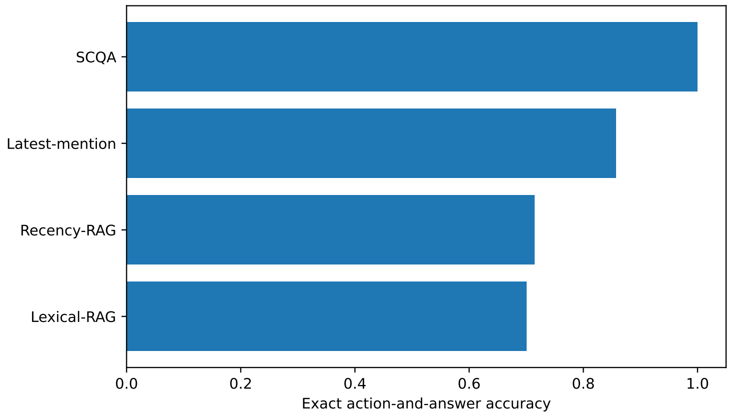

Fig. 3 Oracle-state diagnostic accuracy. The result demonstrates that the specified invariants are executable. It does not estimate end-to-end LLM accuracy because situation variables are supplied rather than inferred.

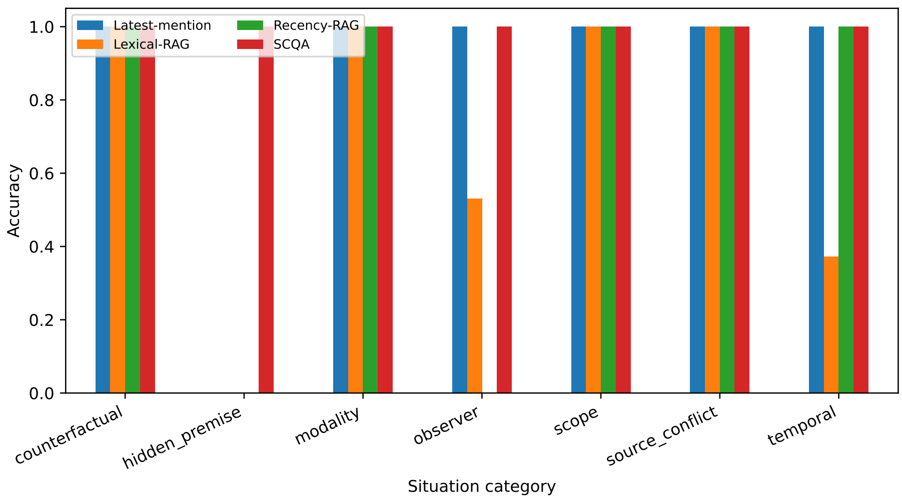

Fig. 4 Accuracy by diagnostic category. Each category isolates one applicability variable, enabling mechanism-level failure localization.

study. It shows that the implementation does not merely benefit from a generic instruction to be cautious; behaviour changes when the specific variable required by an item is removed. Because benchmark and solver share the same schema, the result is best viewed as a mechanistic unit test that future neural systems should pass under both gold and predicted states.

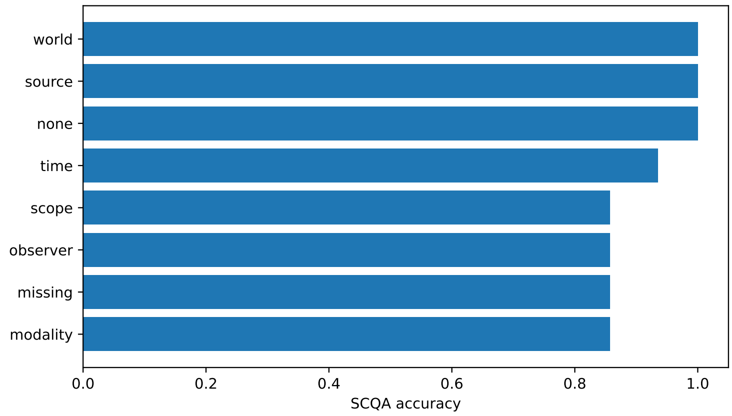

Fig. 5 Oracle-state component ablations. Each removed variable recovers the predicted category-specific failure, providing a causal software diagnostic.

Table 2 Claim-level accuracy of the deterministic text sensor ($n = 7{,}800$ claims per slot).

Slot	Actor	Event	Time	Validity	Scope	Modality	Source	Observer	World
Accuracy (%)	30.8	92.3	0.0	0.0	61.5	100.0	30.8	92.3	100.0

Predicted states expose a sensing bottleneck

We removed access to the structured claim fields and reconstructed a restricted situation state from claim text using the released deterministic sensor. Across all 4,200 diagnostic instances, action-and-answer accuracy fell from 100.0% with gold states to 77.9% with predicted states. Deliberately overwriting source status, modality and scope in the predicted state reduced accuracy further to 35.1%. These are complete executions, not projected values. They show that the oracle result is not preserved when situation sensing is imperfect. Because the texts are generated from the same seven scenario families and the sensor is a transparent rule baseline, these results remain controlled diagnostics rather than evidence of natural-language or LLM generalization.

Claim-level scoring localizes the sensor's limitations (Table 2). Event type reached 92.3%, modality 100.0%, scope 61.5%, actor 30.8% and joint source/status 30.8%. Temporal expressions and validity intervals scored 0% because the generated claim strings omit the numeric times stored in the hidden schema: a system cannot reconstruct a state variable that is neither expressed nor retrieved. Observer and world slots reached 92.3% and 100.0% under repetitive generated language and are not estimates of natural extraction. In this deterministic pipeline, 927 final errors are attributable to sensing, zero to update/policy under gold state, and generation is a fixed renderer.

Table 3 Evidence ladder and permitted interpretation.

Evidence level	What is available	Permitted claim
E1: formal definition	Situation schema and applicability relation	The failure class is operationally testable
E2: oracle unit tests	4,200 generated instances and matched ablations	Specified invariants are executable and causally separable
E3: natural prevalence	Independent temporal/situated QA literature	Situation dependence occurs in natural questions
E4: predicted-state control	Deterministic text sensor on 4,200 generated items	Sensing errors reduce controlled end-to-end accuracy; natural-text generalization remains open
E5: multi-model intervention	Execution harness only; no successful model responses	Needed to claim LLM improvement and generality
E6: deployment / human study	Not completed	Needed for operational safety and human comparison

Natural-language benchmarks establish prevalence, not SCQA efficacy

Independent resources show that situation dependence is not confined to the generated benchmark. SituatedQA reported that approximately 16.5% of sampled NQ-Open questions depended on time or place and that updated evidence did not remove a substantial present-time performance gap [5]. FreshQA, PAT-Questions, TimeQA, ChronoQA and TDBench test related problems in freshness, evolving facts and temporal validity [11–15]. These studies motivate the problem definition, but they are not evaluations of SCQA and are not counted as evidence that the proposed intervention improves a frontier model.

An evidence ladder separates established findings from open claims

Table 3 makes the claim boundary explicit. The current study establishes that seven applicability invariants can be represented, executed and independently ablated under oracle structured states. It supports, but does not establish, the hypothesis that explicit situation states will reduce analogous errors in LLMs. End-to-end efficacy requires matched multi-model experiments on natural text, predicted-state evaluation, strong prompt/RAG/agent baselines, clustered uncertainty estimates, human annotation and efficiency analysis. We release the adapter, schemas and reporting protocol for these experiments but do not substitute unexecuted predictions for results.

Discussion

The core insight is that relevance is not applicability. Retrieval can return a well-supported statement while a system still applies it to the wrong time, scope, epistemic state or possible world. Situation engineering turns this failure from an informal complaint about "missing context" into a set of state variables, update rules and behavioural consequences that can be tested separately.

A complementary layer, not a replacement vocabulary

Prompt, context, retrieval, memory, tool, harness, evaluation, security and serving engineering retain distinct responsibilities. Prompts specify the task; context and retrieval supply evidence; memory preserves observations; tools provide capabilities; harnesses govern workflow and permissions; evaluations measure behaviour; serving systems manage resource constraints. Situation engineering owns the query-relative semantics of the active state and the propagation of unresolved applicability into policy. A useful contract is

$$(C_t, M_t, O_t, H_t) \to (\mathcal{S}_t, U_t, P_t), \tag{5}$$

where context, memory, tool observations and harness traces produce candidate situations $\mathcal{S}_t$, unresolved variables U_t and the permitted policy P_t.

This contract prevents conceptual inflation. Retrieval quality should not be renamed situation engineering, and a JSON field named "situation" is not evidence of situation awareness. A system must show that state representation causally changes behaviour in predicted cases, that state uncertainty affects answering or action, and that failures can be assigned to sensing, update, validation, policy or generation.

Implications for hallucination and agent evaluation

A generated claim can be textually entailed by a document yet wrong for the query. Evaluation should therefore record both support and applicability. In free-form generation, claims should be linked to the situation state and provenance on which they depend. In agents, tools should expose preconditions, state transitions and failure semantics rather than only free-text outputs; harnesses should block consequential actions while answer-critical state is unresolved. Evaluation engineering becomes the sensor layer that checks each transition and not merely the final answer.

The response policy is also part of correctness. Clarification and abstention are not failures when the situation is under-specified. Conversely, excessive clarification is costly and can make systems unusable. Future evaluations should therefore report answer accuracy, clarification utility, unnecessary-question rate, abstention precision, selective risk, coverage, latency, token use and monetary cost. These measures make explicit the trade-off between semantic verification and serving efficiency.

What is established, supported and proposed

This study establishes a formal error class, an interoperable state schema, a reference implementation and matched oracle-state ablations. Independent literature supports the premise that time- and place-dependent questions are common enough to matter. The study does not establish that current frontier models exhibit every proposed category at a particular frequency, nor that SCQA improves them under natural language. Those are hypotheses enabled by the released protocol, not completed results.

This distinction changes the scientific role of the paper. The contribution is not a claim of a production-ready solution. It is a research scaffold that allows subsequent work to ask more precise questions: Is the dominant error in state sensing or state update? Does explicit state outperform a matched structured prompt? Does a temporal graph add value after controlling for token budget? When should a system ask rather than retrieve? How robust is a state to adversarially stale evidence? Which costs arise from maintaining provenance and multiple candidate worlds?

A staged research programme

We propose six stages for cumulative evidence. Stage 1 uses formal and synthetic unit tests to verify semantics. Stage 2 evaluates natural-language situation sensing with slot, event, temporal, scope, modality, observer and world-state metrics. Stage 3 performs matched interventions on several model families while holding evidence and computation budgets constant. Stage 4 tests compositional and distributional generalization: unseen templates, longer event chains, nested scope, conflicting authoritative sources, multiple observers, multilingual data and multimodal observations. Stage 5 adds human annotation and interaction to measure clarification and abstention. Stage 6 evaluates long-running agents, state drift, security attacks, latency, cost and independent auditing.

The SE0–SE4 maturity model and longer research agenda are moved to the Supplementary Information. The present implementation reaches explicit state-and-policy control only for seven supplied variables.

Limitations and risks

The diagnostic data are generated from a schema shared with the solver, so benchmark-solver circularity is intentional and limits external validity. The item count overstates statistical independence because examples share template families. The baselines are software controls, not state-of-the-art LLM systems. Confidence values are rule-derived and do not constitute neural calibration. No human participants, multilingual evaluation, multimodal sensing, frontier-model calls or deployed-agent study were completed. These limitations are not resolved by additional prose and require new experiments.

Explicit state can also create new attack surfaces. An adversary may manipulate timestamps, source authority, identity, scope or world labels. A centralized state representation may expose sensitive user information or create false confidence in a flawed schema. Situation-engineered systems therefore require provenance, access control, privacy-preserving memory, conflict visibility and independent evaluation. “Explicit” must not be confused with “correct”.

Conclusion

Situation engineering provides a bounded and testable way to connect several communities working on reliable AI. Its durable contribution will depend not on the adoption of a new label, but on whether researchers use the interfaces to produce stronger measurements, interoperable state representations and systems that know when available evidence does and does not apply. The present artefact is intended as the first rung of that evidence ladder: a transparent formal and software scaffold on which natural-language, multi-model and deployment studies can be built.

Methods

Problem formulation

An item consists of a query q, evidence claims E, a query time and, where relevant, an observer or possible-world identifier. A claim may be supported in isolation but inapplicable to the query. We define

$$\mathrm{Apply}(e, q, S) \in \{\mathrm{valid}, \mathrm{invalid}, \mathrm{unresolved}\}, \tag{6}$$

where S is the active situation state. The system first constructs or receives S, then updates competing claims and selects an action from answering, retrieval, clarification, abstention or task-specific action.

The end-to-end system is decomposed as

$$(q, E) \xrightarrow{f_{sense}} \hat{S} \xrightarrow{f_{update}} \hat{S}' \xrightarrow{\pi} a \xrightarrow{g} y. \tag{7}$$

This decomposition supports stage-specific evaluation. The gold-state condition supplies S directly and evaluates f_{update}, π and the deterministic answer renderer. A predicted-state condition applies a text-only deterministic sensor before the same update and policy stages. It evaluates composition on generated claim language, but not f_{sense} on unrestricted natural language.

Situation schema

The schema contains actors and entities; events and state transitions; valid and observation time; scope; modality and source status; observer-specific knowledge; possible-world identity; provenance; and unresolved answer-critical variables. Update rules include temporal supersession, cancellation, source-authority preference where specified, scope containment, observer visibility and world separation. Unresolved variables are preserved rather than converted into a guessed binary state.

SituationCatch-Bench generation

The deterministic generator creates 600 instances for each of seven categories using seeded lexical variation and shuffled claim order. Each item includes the question, claims, structured situation fields, gold action, gold answer aliases and category. The categories and gold semantics are documented in the bundled datasheet. Generation and evaluation are intentionally co-designed: the benchmark is a conformance suite for explicit semantics, analogous to unit tests for a software protocol, rather than an independent estimate of natural-language performance.

Future benchmark versions should use separate authors for data construction and system development, clustered scenario families, hidden generators, natural and semi-synthetic sources, adjudicated annotations, compositional splits and adversarial state manipulations. Recommended natural-data fields and annotation instructions are supplied in the Supplementary Information.

Reference implementation

SCQA is a deterministic Python reference implementation with no trained parameters. It parses the supplied structured fields, performs category-specific state updates and selects a permitted response. The three software-control baselines select claims using lexical overlap, recency or latest mention and do not maintain the complete situation state. These controls test whether the specified state variables are necessary for the generated invariants; they are not intended to represent competitive LLM, RAG or agent baselines.

Predicted- and corrupted-state evaluation

The predicted-state sensor receives only claim text and infers event type, actor, temporal expression, modality, source status and world or scope using documented deterministic patterns; it cannot copy hidden gold slots. The downstream engine is unchanged. In a pre-specified corruption condition, predicted source status is replaced by a generic reported source, modality is forced to confirmed and scope is forced to global. All three conditions contain the same 4,200 identifiers and are scored by exact action and answer. Item-level and aggregate outputs are released in `results/state_conditions.csv` and `results/state_conditions_summary.csv`. The sensor is additionally scored claim by claim for actor, event, temporal expression, validity interval, scope, modality, joint source/status, observer-knowledge and world slots. Because claim order is preserved, this comparison does not require learned alignment. The resulting files are `results/predicted_state_slots.csv` and its aggregate summary.

Metrics and analysis

The primary diagnostic measure is exact action-and-answer accuracy. For items requiring clarification, correctness requires selecting clarification rather than providing the latent answer. Category-level accuracy and matched component ablations localize failure mechanisms. Because instances share template families, item-level confidence intervals and P values are not used as primary scientific evidence in the revised manuscript. The released scripts retain item-level summaries for regression testing, but future natural-language studies should use scenario- or template-clustered bootstrap intervals, paired tests, multiple-comparison control and pre-specified primary outcomes.

Rule-derived confidence values from the original implementation are not treated as neural calibration. End-to-end work should report Brier score, expected calibration error, selective risk, risk–coverage area, clarification utility and abstention precision/recall using model- or state-derived probabilities.

Ablation design

Seven ablations independently remove temporal validity, source status, modality, scope, observer knowledge, world identity or missing-variable detection. Each ablation is applied to all items, and its expected effect is specified before execution. The purpose is to test causal alignment between representation and failure category, not to estimate an average treatment effect in natural text.

Protocol for end-to-end multi-model evaluation

A confirmatory study should compare at least the following matched conditions: direct prompting; chain-of-thought or structured prompting; date-aware context; standard RAG; temporal or event-aware RAG; reflection or verification agent; and Situation Engineering. The generation model, evidence set, retrieval corpus, maximum context, output budget and sampling parameters should be held constant where the design permits. At least three proprietary and three open-weight model families should be evaluated using immutable model identifiers.

The primary intervention contrast must be *same model + same evidence + same computation budget* with versus without the explicit situation layer. A factorial analysis should independently vary prompt, context/retrieval, harness and situation state. Competitive conditions should include long-context prompting, few-shot structured reasoning, self-consistency, self-refinement, Self-RAG, graph or temporal retrieval, dialogue/belief-state tracking, clarification and calibrated abstention.

For each call, researchers should archive the prompt, evidence, raw response, normalized answer, action decision, model identifier, provider, execution date, latency, token use, monetary cost, errors and retries. Evaluation must report gold-state, predicted-state and deliberately corrupted-state conditions. The predicted-state condition should score entity/event extraction, temporal relations, validity intervals, scope, modality, source status, observer knowledge, possible-world identity and final answer. This enables error attribution to sensing, updating, policy or generation.

The bundled `code/run_multimodel_eval.py` provides provider adapters and refuses to create placeholder outputs when credentials are absent. No frontier-model performance values are reported because such calls were not executed in the study environment. The script is an executable protocol, not a completed experiment.

Generalization and efficiency protocol

External evaluation should pre-register held-out scenario families and test unseen templates, longer event chains, nested scopes, conflicting authoritative sources, multiple observers, multilingual questions and adversarial stale evidence. Splits must occur at scenario or generator-grammar level rather than over lexical variants. Uncertainty intervals should use scenario-clustered resampling. Each condition must report latency, input and output tokens, retrieval and tool calls, retries, monetary cost and clarification overhead. Accuracy gains without matched resource reporting cannot identify the effect of the situation layer.

Human and deployment evaluation

Human evaluation should use at least three independent annotators per natural item, blinded to method, with a documented adjudication procedure. Recommended outcomes include state-slot agreement, answer correctness, appropriateness of clarification or abstention, unnecessary clarification and perceived justification quality. Studies involving participants should report ethics approval or exemption, consent, compensation and privacy handling.

The released annotation program generates separately shuffled, blinded packets for three or more annotators. It requires complete independent labels before computing Fleiss' κ and unanimous agreement for action, answer and state slots; incomplete packets terminate with an error. Disagreements are adjudicated only after independent files are frozen. No human agreement value is reported because independent human annotations were not collected.

Deployment studies should additionally measure state drift, invalidation latency, recovery after contradictory observations, action-precondition violations, security attacks on state metadata, throughput, tail latency, energy, token use and cost. The state and provenance required to audit a consequential decision should be retained subject to privacy and access-control requirements.

Reproducibility and reporting protocol

The package contains the generator, generated benchmark, reference implementation, predictions, figures, datasheet and exact commands. To make future systems comparable, authors should report: (1) the situation schema; (2) which variables are supplied, retrieved or inferred; (3) update and conflict semantics; (4) response and action policies; (5) provenance and observability; (6) stage-specific errors; (7) prompt, context, retrieval, memory, tool and harness configurations; and (8) resource costs. A machine-readable experiment manifest is recommended.

Ethics and use of generative AI

The controlled benchmark contains no personal records or human participants. Generative AI assisted drafting and software development. The author inspected the code, generated outputs, calculations and references and remains responsible for the manuscript. No empirical values were fabricated to represent unexecuted model calls or human studies.

Data availability

SituationCatch-Bench v0.1 is included in the public source repository at https://github.com/leemgs/sage and in the submission package as `data/situationcatch_bench.jsonl`, together with its deterministic generator, datasheet and item-level outputs. This package is sufficient to reproduce the controlled results. The exact reviewed version is identified by the Git commit attached to the submission. An immutable archival release, an institutionally approved licence and DOI remain required before acceptance; no placeholder DOI is represented as an existing deposit.

Code availability

The complete executable implementation is included under `code/`. Running `PYTHONPATH=code python code/run_experiments.py` regenerates gold-, predicted- and corrupted-state outputs without network access or proprietary dependencies. The multi-model adapter records raw responses and errors and never creates synthetic API results. The annotation program requires three complete independent packets before reporting agreement. Source and item-level outputs are also available at https://github.com/leemgs/sage. The repository README records exact commands; `SHA256SUMS.txt` records packaged artefact checksums. A public immutable release and archival DOI remain pre-acceptance requirements.

Author Contributions

G.L.: Conceptualization, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

Competing Interests

The author declares no competing interests.

Acknowledgements

The author thanks colleagues who provided early discussions on reliable language-model reasoning. No external funding is declared in this draft.

References

[1] Lewis, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Adv. Neural Inf. Process. Syst.* **33**, 9459–9474 (2020). https://arxiv.org/abs/2005.11401

[2] Asai, A., Wu, Z., Wang, Y., Sil, A. & Hajishirzi, H. Self-RAG: learning to retrieve, generate, and critique through self-reflection. In *Proc. International Conference on Learning Representations* (2024). https://openreview.net/forum?id=hSyW5go0v8

[3] Yao, S. et al. ReAct: synergizing reasoning and acting in language models. In *Proc. International Conference on Learning Representations* (2023). https://openreview.net/forum?id=WE_vluYUL-X

[4] Schick, T. et al. Toolformer: language models can teach themselves to use tools. *Adv. Neural Inf. Process. Syst.* **36** (2023). https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html

[5] Zhang, M. J. Q. & Choi, E. SituatedQA: incorporating extra-linguistic contexts into QA. In *Proc. EMNLP*, 7371–7387 (2021). https://aclanthology.org/2021.emnlp-main.586/

[6] Allen, J. F. Maintaining knowledge about temporal intervals. *Commun. ACM* **26**, 832–843 (1983). https://doi.org/10.1145/182.358434

[7] Mei, L. et al. A survey of context engineering for large language models. *Preprint at* arXiv (2025). https://arxiv.org/abs/2507.13334

[8] Zhong, H. & Zhu, S. AI harness engineering: a runtime substrate for foundation-model software agents. *Preprint at* arXiv (2026). https://arxiv.org/abs/2605.13357

[9] Lin, J. et al. Agentic harness engineering: observability-driven automatic evolution of coding-agent harnesses. *Preprint at* arXiv (2026). https://arxiv.org/abs/2604.25850

[10] Barwise, J. & Perry, J. *Situations and Attitudes* (MIT Press, 1983). https://mitpress.mit.edu/9780262521093/situations-and-attitudes/

[11] Vu, T. et al. FreshLLMs: refreshing large language models with search engine augmentation. *Preprint at* arXiv (2023). https://arxiv.org/abs/2310.03214

[12] Meem, J. et al. PAT-Questions: a self-updating benchmark for present-anchored temporal question answering. In *Findings of ACL* (2024). https://aclanthology.org/2024.findings-acl.777/

[13] Chen, W., Wang, X., Wang, W. Y. & Chen, W. A dataset for answering time-sensitive questions. In *NeurIPS Datasets and Benchmarks* (2021). https://openreview.net/forum?id=Mn1lWz3F4x

[14] Chen, Z. et al. ChronoQA: a question answering dataset for temporal-sensitive retrieval-augmented generation. *Sci. Data* (2025). https://www.nature.com/articles/s41597-025-05757-2

[15] Kim, S., Wang, J., Xie, X. & Whang, S. E. Harnessing temporal databases for systematic evaluation of factual time-sensitive question answering in large language models. *Preprint at* arXiv (2025). https://arxiv.org/abs/2508.02045

[16] van Benthem, J. Dynamic logic for belief revision. *J. Appl. Non-Classical Logics* **6**, 129–155 (1996). https://doi.org/10.1080/11663081.1996.10510843

[17] Stalnaker, R. Common ground. *Linguist. Philos.* **25**, 701–721 (2002). https://doi.org/10.1023/A:1020867916902

[18] Alchourrón, C. E., Gärdenfors, P. & Makinson, D. On the logic of theory change: partial meet contraction and revision functions. *J. Symb. Logic* **50**, 510–530 (1985). https://doi.org/10.2307/2274239

[19] Doyle, J. A truth maintenance system. *Artif. Intell.* **12**, 231–272 (1979). https://doi.org/10.1016/0004-3702(79)90008-0

[20] Kaelbling, L. P., Littman, M. L. & Cassandra, A. R. Planning and acting in partially observable stochastic domains. *Artif. Intell.* **101**, 99–134 (1998). https://doi.org/10.1016/S0004-3702(98)00023-X

[21] Henderson, M., Thomson, B. & Williams, J. D. The second dialog state tracking challenge. In *Proc. SIGDIAL*, 263–272 (2014). https://aclanthology.org/W14-4337/
[22] Snodgrass, R. T. Temporal databases. In *Theories and Methods of Spatio-Temporal Reasoning in Geographic Space*, 22–64 (Springer, 1992). https://doi.org/10.1007/3-540-55966-3_3
[23] Kwon, W. et al. Efficient memory management for large language model serving with PagedAttention. In *Proc. ACM SOSP*, 611–626 (2023). https://arxiv.org/abs/2309.06180
[24] McCarthy, J. & Hayes, P. J. Some philosophical problems from the standpoint of artificial intelligence. In *Machine Intelligence 4*, 463–502 (1969). https://www-formal.stanford.edu/jmc/mcchay69.pdf
[25] Kowalski, R. & Sergot, M. A logic-based calculus of events. *New Gener. Comput.* **4**, 67–95 (1986). https://doi.org/10.1007/BF03037383
[26] Reiter, R. *Knowledge in Action: Logical Foundations for Specifying and Implementing Dynamical Systems* (MIT Press, 2001). https://mitpress.mit.edu/9780262681495/knowledge-in-action/

AI-use disclosure

OpenAI Codex, based on GPT-5, assisted English-language drafting, editorial restructuring, Python code scaffolding and test generation. An image-generation model produced the repository logo, which is not scientific evidence. The author reviewed every modified manuscript section and source file, executed the tests and controlled measurements, and checked bibliographic titles and destination URLs against publisher, proceedings or preprint pages. Generative AI was not treated as an author or annotator and did not supply human labels, model responses or unexecuted empirical measurements.